High Performance Computing Co-Design Space III : Simulation Design Space and Optimization Design Space

Sudharshan Srinivasan, Anirudh Seshadri, Dhanasekar Sundararaman

In consolidating innovative concepts for designing the exascale computing systems, the application complexity design space, heterogeneous many-core architecture design space and parallel programming language design space, all get integrated through a binding process, including the simulator and optimizer design spaces. An ensemble of these design spaces is what is called many-core- co-design- spaces. These are conceptually deeply characterized in three companion papers including this paper establishing the fact that the prospective co- design spaces achieve excellent power-performance efficiency in realizing the target heterogeneous many-core architecture for simultaneous execution of co-applications without space time sharing, (which is called ALL Embracing Execution ALLEE ALLDE) over the traditional approaches driven by the multi-programming. Simultaneous execution of co-applications in heterogeneous many-core architecture without space time sharing is a concept in which instructions of the same type but from different applications get simultaneously pipelined within the same functional unit. This is unlike the multi-programming and augments the resource utilization to a large extent and in such an environment the main memory bank allocation and the cache policies are tailored to be aware of the dynamics of co-applications execution to achieve further memory efficiency. Traditional heterogeneous many-core simulators are incapable of assimilating these concepts.None of the current simulators are good enough to get merged with the proposed many-core co-design spaces and the reasons for the same are explained in this paper where in such a simulator is proposed. Design time power-performance optimization of many-core architectures essentially needs to take care of simultaneous execution of co-applications and not just a single application, unlike the current optimizers. Such an optimizer involves three levels of optimization one optimizing the overall count of functional units and storage elements, second optimizing the types of core and the third the overall core count.This paper proposes such an optimizer. Using the proposed simulator and the optimizer, very extensive results pertaining to the design of power-performance optimized heterogeneous many core architecture targeting a set of four co-applications, built using complex kernels of practical consequences, are presented in this paper. These results clearly exhibit that the many-core co-design spaces and the associated innovative concepts achieve far better power-performance efficiency in envisaging the target heterogeneous many-core architecture driven by the ALLDE model over the long established multi-program driven architectures.

***Index Terms*—Optimization, Simulation, Machine Learning.**

I. Introduction

Several research project reports and papers have been published, advocating co-design methodology for developing low power high-performance computing systems. The details of these research works are highlighted in the companion paper I[paper] in the context of the proposed many-core co-design spaces, establishing the advantages and the dominance of the later over the former with regard to the design and development of heterogeneous many-core architecture. These research works no doubt meticulously focus on every aspect of co-design principles associated with application, software and many-core architecture, stressing more on memory system hierarchy and low power technology down to 10nm.In fact most of the these reports stress more on technology which is the paramount driving force. There are quite a number of application specific co-design based research projects [paper1], but they are isolated, lacking some fundamental generic approach towards accomplishing power-performance efficient heterogeneous many-core architectures. None of these research reports and papers[paper1] have come out with an integrated solution platform for co-design methodology to characterize, application complexity design space, heterogeneous many-core architecture design space, parallel programming language design space, simulator design space and optimizer design space and to put all together through a binding process under the co-design automation. An ensemble of these spaces is called many-core co-design spaces.

Many-core co-design spaces are portrayed in Fig.1, conceptualizing the integration of the application complexity design space, the heterogeneous many-core architecture design space, parallel programming language design space, the simulator design space and the optimizer design space, for designing heterogeneous many-core architecture along with the correlated ISA, both tailored to the computational and communication structures of multiple applications.Many-core co-design spaces shown in Fig.1 are reinforced by several innovative and simple concepts which are elucidated in the companion paper I [1]. Among them the most important being the conception of simultaneous execution of co-applications without space time sharing[paper1].Here instructions of same type but from different applications get simultaneously pipelined within the same functional unit unlike the multiprogramming, to make more effective use of many-core resources and presented in the companion paper I [paper -1].This type of execution is termed, All Embracing Execution-ALLEE- ALLD(Double)E-ALLDE and this concept has lead to an innovative programming paradigm as explained in the companion paper[paper1].The design of heterogeneous many-core architecture for ALLDE, using many-core co-design spaces is established, adopting an innovative core structure conceptualized in the companion paper I [paper 1], made of disparate hardware blocks that are entirely distinct from the conventional in order and out of order core structures on which the existing simulators

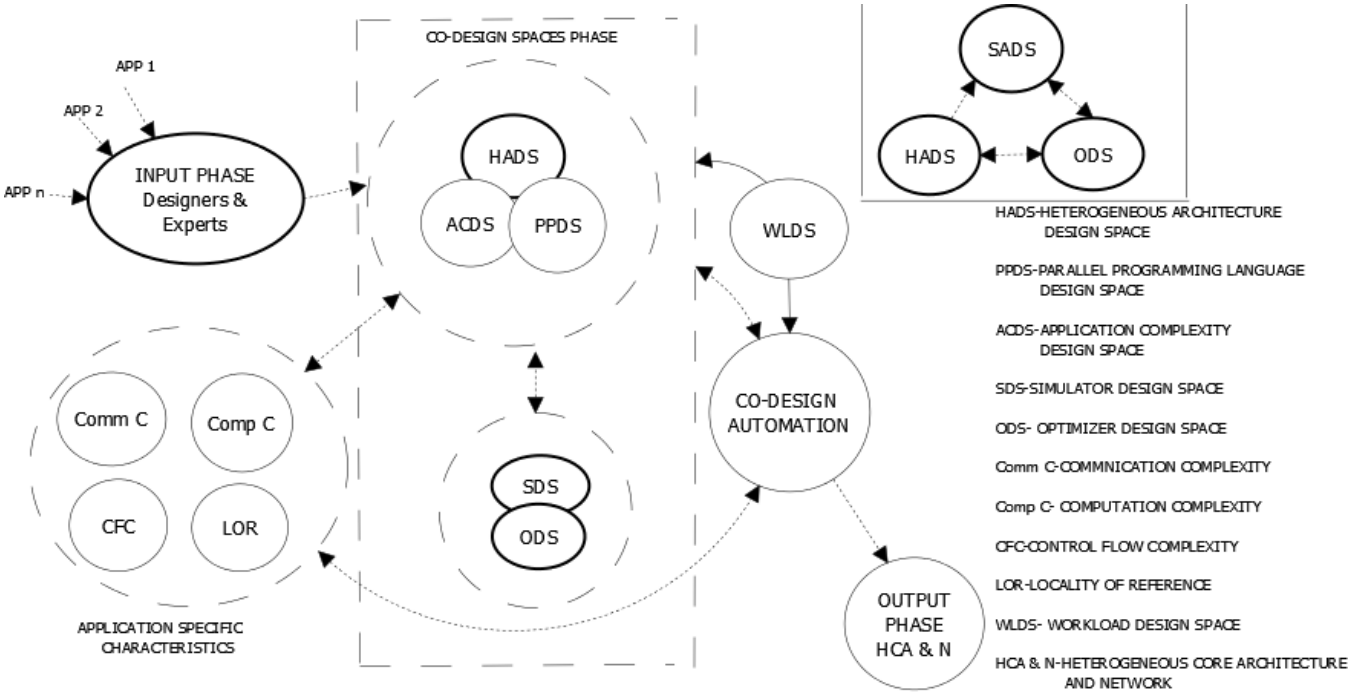

Fig. 1. *Result section flow*

are based. Further the concept of core migration can get substantially eliminated due to ALLDE and is well described in the companion paper I[paper1].The attributes of the application complexity design space, the heterogeneous many-core architecture design space and the parallel programming language design space and their inter-relationships are well expounded in the companion paper [paper1,paper II].The papers and research project reports [paper I] dont deal with simulators and optimizers in the mind set of many-core co-design spaces driven by ALLDE.

The fact is that the many-core co-design spaces lead to an inherent formation of different core types, intra core components and power-performance optimized over all count of cores in conformation with heterogeneous characteristics of the co-applications. The excellent impact of the many-core co-design spaces and the associated simple concepts, on the design and development of power-performance efficient heterogeneous many-core architecture for ALLDE, over the traditional design approaches and the multi-programming, is established in this paper based on an extensive simulation and optimization carried out on the target many- core architecture for ALLDE on a set of complex workloads. These workloads are generated following the theory of customizable workload generation in particular for benchmarking heterogeneous many-core architecture the tool set in the companion paper II[paper 2].The many fold advantages and the dominance of the customizable workload generation model over SPEC, LINPACK,SPLASH2[],PARSEC[],and IBS are well accentuated in the companion paper II[paper2].

The simulator design space, the optimizer design space and their interactions with the heterogeneous many-core architecture design space are exemplifiedin this paper. Refer to the bolded portion of Fig.1.Heterogeneous many-core architecture design space, is the warehouse of widely varying virtual hardware components, like register files(buffers),cache components, different algorithm level functional units(pertaining to numeric, semi-numeric, non-numeric operations),scalars, on core network and inter-core network elements. These virtual components are in the form of time and power models. Based on the attributes (computational structure, communication complexity, control complexity) of co-applications, the simulator is configured with the help of heterogeneous many-core architecture design space and the parallel programming language design space.

Design-time power-performance optimization of heterogeneous many-core architectures, essentially need to take care of ALLDE and not just a single application unlike the current optimizers.It is shown in the subsequently that such optimizer, irrespective of the optimization algorithms, needs to perform three levels of optimization to achieve power performance efficiency for the target heterogeneous many-core architecture and the correlated ISA, to take into account the impact ALLDE. None of the current simulators and design time optimizers are competent enough to get merged with the concept of many-core co-design spaces for co-applications. In this paper such a simulator and an optimizer are proposed.

The section II characterizes the simulator in the framework of many-core co-design spaces highlighting the drawbacks of the traditional simulators in this context.

II. SIMULATOR:MANY-CORE CO-DESIGN SPACES PERSPECTIVE

Simulation is an integral part of the design process for high performance heterogeneous many-core computing systems. Design and development of these systems are complexand cost intensive. This means that the design needs to be validated by repeated simulation cycles.

Taking cognizant of the current background of high performance computing research [moreexascale ref] several new concepts are proposed in the companion paper I [paper 1] to accomplish the binding process by means of many-core co-design spaces, for designing better power-performance efficient heterogeneous many-core architecture for ALLDE.Hence the simulator design needs to be in conformity with the many-core co-design spaces and the associated innovative concepts which are briefed below.
In ALLDE instructions of the same type either independent or dependent (intra application) from multiple applications can get simultaneously pipelined within the same functional unit. The ALLDE concept greatly augments the resource utilization unlike the multi-programming, and this is established by extensive simulation results presented in section IX of this paper. Further the companion paper I [1] elucidates the necessity of both themain memory bank allocation and the cache replacement policies to be aware of the dynamics of multiple applications execution(ALLDE) to achieve overall memory system efficiency.Also a design space paradigm for parallel programming languages has been proposed[paper1] to build new constructs specific to the given set of multiple applications and the corresponding target heterogeneous many core architecture which itself is specific to computational structures and the communication complexity of multiple applications put together. The mechanism of many-core co-design spaces is bound to crop up different ISAs, tightly coupled to the attributes of multiple applications and the corresponding target many-core architecture. To be precise these ISA sets are from the parallel language design space deliberated in the companion paper I [1].

A. *Current many-core HW/SW Simulators:what is lacking ?*

The current HW/SW simulators are fundamentally not designed from the standpoint of many-core co-design spaces. The current simulators lack the following.

1) Cannot support ALLDE. But majority of them support multi-programming by incorporating the conventional operating system
2) Cannot simulate multiple applications aware cache replacements policies[paper1 and isvlsi],and this is indispensible in ALLDE.Otherwise the traditional simulators simulate the cache systems comprehensively.
3) Incapable of multiple application aware, dynamic main memory bank allocation.
4) The target simulation is only for the existing ISAs and hence the traditional simulators cannot be integrated with the many-core co-design spaces. In ALLDE the ISA have totally different format to include application ID, instruction ID besides instruction type (opcode).
5) On the whole cannot simulate the core model (completely presented in the companion paper I [paper1]) of the many-core co-design spaces for designing heterogeneous many-core architecture for ALLDE.The core model is briefed in the section IV.

The conclusion is that the current simulators need to undergo changes to get merged with the many-core co-design spaces and the ALLDE.

III. RELATED WORKS: MANY-CORE SIMULATORS

The Fig.2 shows a schematic of the generic class of simulators and their association.

The SimOS[ref] simulator is built for modeling complete computer system and the operating system and hence can take on all application programs.The Embra[ref] works under SimOS and can simulate processors.The TFsim[ref] is a timing first simulator working with Simics,the functional simulator.The GEMS[ref] is an execution driven simulator that combines Simics and TFsim.The M5[ ] models networked systems.The gem5 is focused on architectural modeling and integrates M5 and GEMS. The Graphite is a parallel multi-core simulator built up from the ground. The MARSSx86 is a cycle accurate simulator built on the emulation framework of QEMU.The Sniper is a multi-core parallel simulator supported by Graphite frame work and the Interval Simulator [ ].The Zsim [ ] simulator engages Pin[ ] to implement dynamic binary translation for fast sequential simulation of OOO cores through instruction driven timing models. The instrumentation-based approach is adopted in Zsim[ ], to exclude functional simulation of the x86 and utilizes the host to carry it out and hence augments the speed up of sequential simulation of detailed modelof a core.The PriME can simulate multi-programmed workloads in an MPI environment, using two levels of parallelism one with the host and the other across hosts.

IV. SIMULATOR DESIGN SPACE : CO-DESIGN PERSPECTIVE

The simulator design space is built around the basic components of a special class of core, customizable with respect to technology(device count)[paper 1],multiple applications characteristics, ILP per clock and exclusively designed for many-core co-design spaces to accommodate multiple applications specific, Algorithm Level Functional Units[paper 1] as well as scalars for the purpose of ALLDE. A schematic of the proposed simulator design space is portrayed in Figs 3 and 4.

In this section a concise description of the components of the core, which establishes the simulator design space is given, highlighting their essentiality from the mindset of power-performance efficiency. A comprehensive description of the components of the customizable core and their detailed architectures, emphasizing their power-performance efficiency are elucidated in the companion paper I[paper 1].

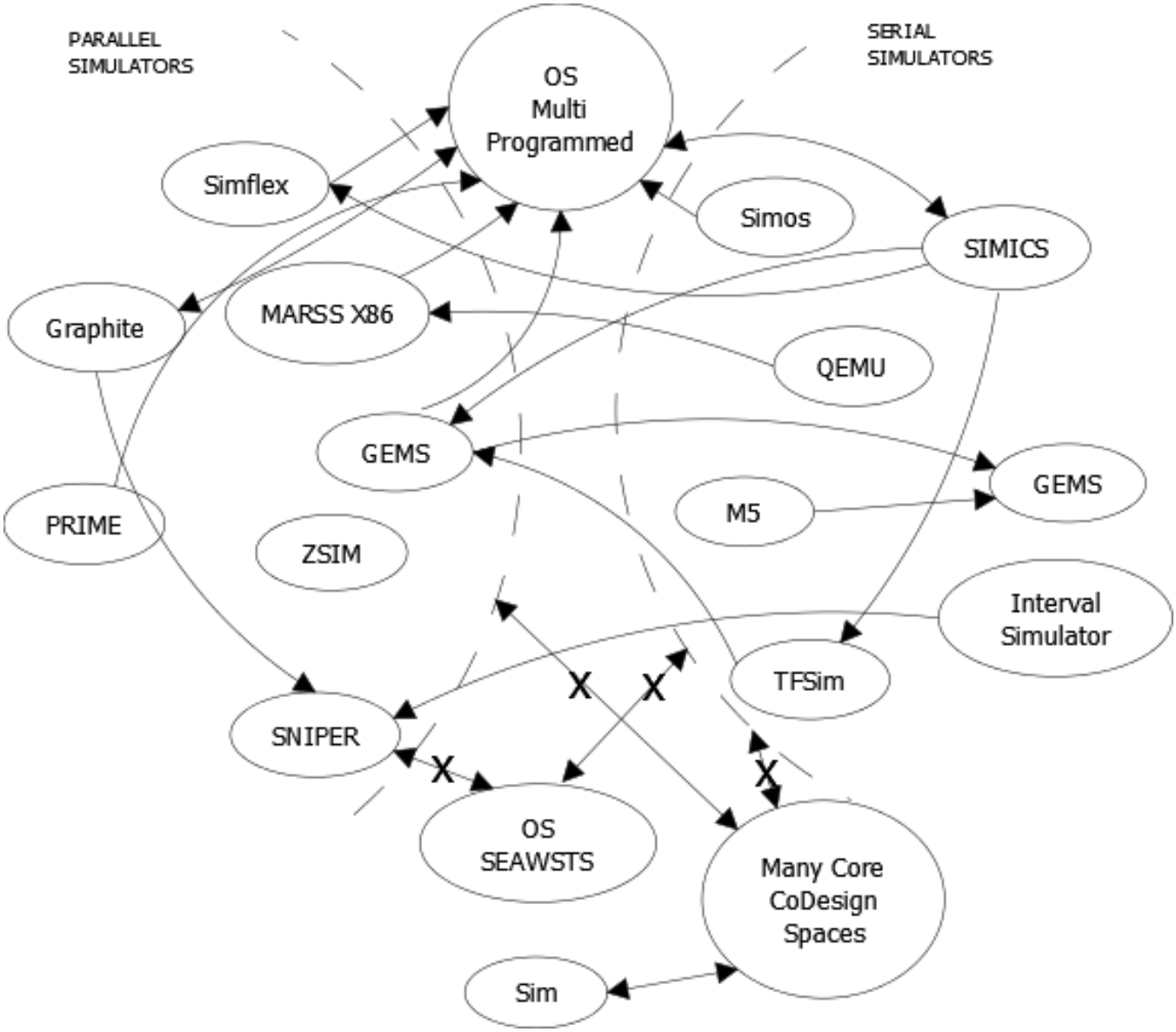

Fig. 2. *Result section flow*

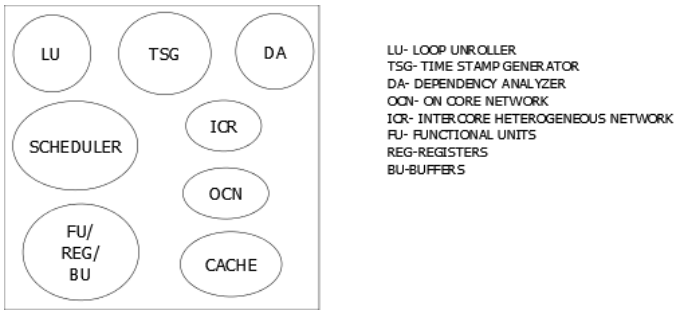

Fig. 3. *Result section flow*

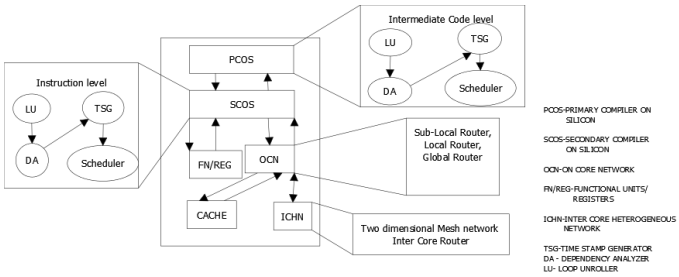

Fig. 4. *Result section flow*

The PCOS performs dependency analysis across the partitions of the incoming intermediate code explained in the companion paper I[paper 1] from the host compiler which is responsible for the syntax and semantic analysis. The PCOS scheduler assigns these intermediate code partitions to the different underlying SCOSs. Dependent partitions are sent to the same SCOSs, simultaneously the count of the instruc-

tions assigned to the different SCOSs are balanced taking cognizant of the depth of dependency across the intermediate code partitions.The activities of SCOSare similar to that of its counterpart, but at much deeper level (instructions) and it controls the dynamics of components[paper I] like loop un-roller, time stamp generator, scheduler etc. It is obvious that the instruction scheduling is committed to the statistics of resources utilization.

V. Simulator XYZ software frame work and software modules

The frame work of the XYZ simulator is shown in Fig.5. To be typed*****

VI. Optimizer: many-core co-design spaces perspective

The many-core co-design spaces methodology of binding application characteristics, architecture characteristics and programming languages as presented in the companion paper I[15] and this paper(refer Fig.1), will help achieve fairly optimal solution during the design cycle itself. This is because, the initial heterogeneous many-core architecture with the correlated ISAs (pre-optimization) areobtained directly from the many-core co-design spaces automation process, presented in the companion paper II [16], based on, computation, control flow and communication complexities (as accentuated in the companion paper I [15]) of multiple applications,specific algorithm level functional units, and both multiple applications and heterogeneous many-core architecture specific parallel language constructs[15].Further the pre-optimization processof co-design automation, (towards designing power-performance optimal heterogeneous many-core architecture) ,creates reduced design space solution, from the view point of simultaneous execution of multiple applications without space time sharing.This will help speed up any optimization algorithms.This pre-optimization process is exemplified in the companion paperII [16].More than the class of optimization algorithm, the methodology of its application in the many-core co-design is important. The extensive simulation results presented in section IX shows the impact of many-coreco-design spaces on design-time power performance optimization over the traditional isolated approaches.These parameters can be further optimized using dynamic power management techniques [reference].

A. Current many-core optimizers: what is lacking

The earliest research paper which looks into power and area optimization during design time for building heterogeneous core based CMPs is [13].Here a methodology for creating a vast sample design space made up of hundreds of cores with varying parameters as listed in table 1 of [13] is given. These cores are classified as in-order and out-of-order, where the former is based on alpha EV(21164) while the latter is based on MIPS R10000. A case study presented in[13], provides a design space consisting of 480 cores having 96 distinct in-order cores and 384 distinct

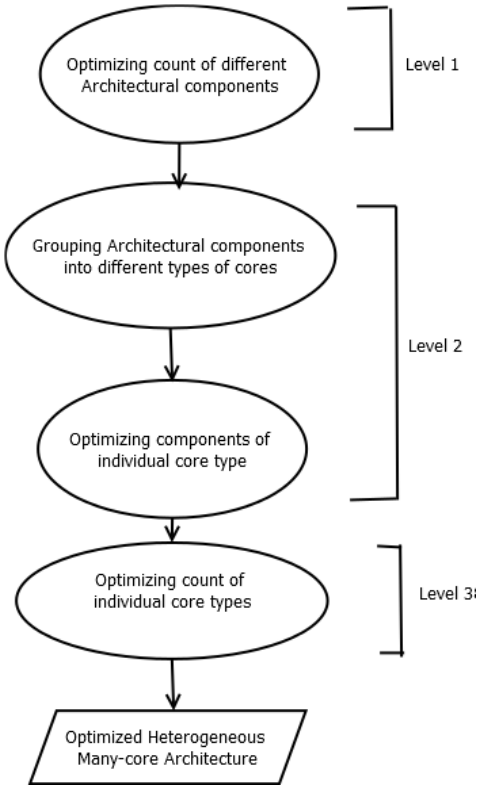

Fig. 5. *Result section flow*

out-of-order with varying parameters are considered for constructing CMPs with 4 distinct cores leading to a sample size of 2.2 billion distinct 4 core CMPs.

Based on the advantages of single-ISA heterogeneous core architectures[14] and [13] one may tend to believe that the power, performance and area optimized solution for 4 core CMPs, can be spotted through efficient search techniques alone, without the knowledge of computation complexity and communication complexity of target applications, appropriate programming model and the associated parallel programming language.

More than the class of optimization algorithm, the methodology of its application in the many-core co-design spaces is important. Here the design-time optimization heuristics are committed to the attributes of the co-applications (computational, communication control flow and complexities) and the many-core architecture (individual count of algorithm level functional units and scalars, specific to computational structures of the multiple applications, on core hierarchical network elements, (like buffer sizes of sub-local, local and global routers, refer Fig.4). Such power-performance optimizer for heterogeneous many-core architectures meant for ALLDE entrenched totally on a different approach which necessarily involves three levels of optimization. This is portrayed in Fig.6.

Level1: In the first phase of optimization, the huge initial count of functional units(multiple applications specific

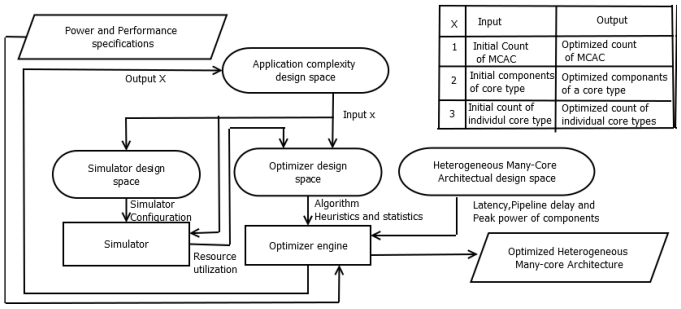

Fig. 6. *Result section flow*

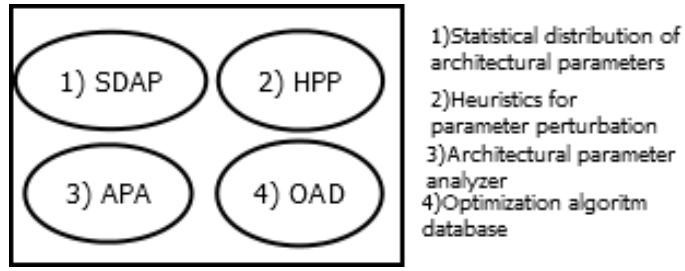

Fig. 7. *Result section flow*

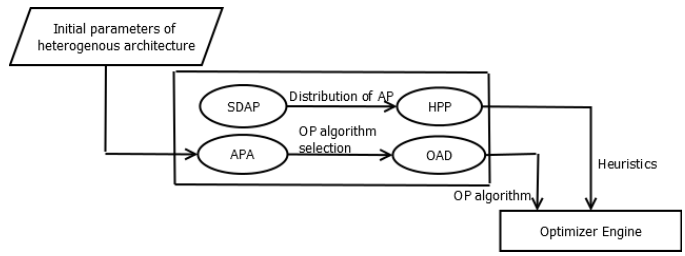

Fig. 8. *Result section flow*

algorithm level functional units and scalars), obtained from the many-core co-design spaces [paper 1] through a simple process of co-design automation , presented in the companion paper II[paper 2],gets power-performance optimized.

Level2: In the second phase of optimization the types of cores get optimized to the desired count, by applying K-means clustering algorithm and the Elbow method, on the power-performance optimized, multiple applications specific algorithm level functional and scalar units space.

Level3: In the third phase the overall count of all types of cores put together gets optimized.

This paper introduces such an optimizer integrated with the proposed simulator

VII. OPTIMIZER DESIGN SPACE: MANY-CORE CO-DESIGN SPACES PERSPECTIVE

The need for constructing the optimizer design space in the context of many-core co-design spaces has been discussed in the introduction .The components of application complexity design space, parallel language design space,heterogeneous core design space are presented extensively in the companion paper II[paper 2][15]. The application complexity design space is built around various classes of numeric ,semi numeric ,non numericalgorithm banks and general purpose operations along with their respective computation and communication complexities which are discussed extensively in the companion paper[15].Along the similar lines, the components of the optimizer design spaceare elucidated considering their respective characteristics.A design space is conceived to provide customizability in order to achieve desired design specifications. Fig.1 presented in section I shows a higher level interaction across different design spaces.

The optimizer design space receives the initial architecturecorresponding to the target multiple applications(for which power-performance efficient heterogeneous many-core architecture is being designed ALLDE) ,in the form of huge set of hardware components, namely algorithm level functional units, scalar units, buffers, registers, latches etc. and their communication patternsas input from application complexity design space. Refer Fig.8.and Fig.1.

It is exemplified in the companion paper II [paper 2] how the relevant initial architecture emerges from the application complexity design space. It is important to note that this architecture is in the form of isolated components rather than core based form. Refer Fig.6. A sample list of these components is shown in Fig.9.

The schematic of the optimization design space is shown in Fig.10.

The modules of the optimizer design space are Statistical Distribution of Architectural Parameters, Heuristics for Architectural Parameter Perturbation, Architectural Parameter Analyzer and Optimization Algorithm Bank. The distribution of architectural parameters is sent from Statistical Distribution of Architectural Parameters module to Heuristics for Architectural Parameter Perturbation Module. This module is responsible for determining the quantum of perturbation of various architectural parameters which are sent to the optimizer engine for implementing the perturbations of specific combinations of parameters. The Architectural Component Analyzer receives the initial optimal parameters of the heterogeneous architecture from application complexity design space. This module selects the appropriate algorithm from the optimization algorithm bank. The selected algorithm is sent to the optimizer engine. The in-depth details of the different modules of optimizer design space are discussed below.

A. The Statistical Distribution of Architectural Parameter

The perturbation of system parameters is fundamental to majority of optimization algorithms and it is performed at each iteration. Perturbation of architectural parameters refers to varying of, cache size at different levels, count of particular type of functional unit, buffer size of on-core network routers [15] and other related parameters. The sample parameter perturbations are shown along with the table.

The Table 1 gives details about perturbation of architectural parameters which are received from the optimizer engine.

A partition refers to a set of consecutive iterations. The mean and variance of different architectural parameters after

perturbation, under each partition areobtained during the optimization process. The size of partitions determines the accuracy of the distribution. A sample list is shown in Table2. To obtain a more accurate track of perturbations, distributions of the mean itself is applied(mean of mean) and this information is sent to the Heuristics for Parameter Perturbation module.

B. Heuristics for Parameter Perturbation

Based on statistical information received, Heuristics for Parameter Perturbation module determines the combination of parameters for which the perturbations are to carried out, and to what extent. These details are sent to the optimizer engine. The heuristics that carried out by this module is shown in Fig.11.

1) Number of parameters = n
2) nCr combinations are available to perturb all combinations of n. These perturbations are performed after initial iterations
3) Initially the parameters of nCr are chosen by varying r randomly.
4) Each parameter is multiplicatively associated with a weight Wranging from 0 to 1.
5) Classify the different parameters in terms of its expected influence on the iterations, toward the optimal solution.
6) Group the parameters according to low end(16 element sorter) and high end(2*2 Matmul) hardware complexity.
7) Criteria for grouping are such that hardware complexities across groups is uniform
8) During each iteration, the influence of randomly chosen combination towards the optimal solution is analyzed.
9) The mean and variance of each of those combinations are obtained from SDAP.
10) These mean and variance are analyzed to obtain the influence towards optimal sol tobased on which the combinations of parameters to be perturbed for the nextiteration.
11) This process continues.

The Algorithm flow corresponding to the

C. Optimization algorithm selector

The optimization algorithm selector is another module responsible for selecting appropriate optimization algorithms based on the characteristics of the solution space which is a very dominant factor especially if one resorts to design an automated process for choosing either a particular optimization algorithm or a hybrid class. The attributes of solution space that proves to be of concern are its large size and unknown characteristics. An example for a large solution space for heterogeneous core optimization problem is presented in[13], and an example of solution space with unknown characteristics,robotic planetary exploration presented[17]. The initial optimal heterogeneous solution space is of reduced complexity because of following reasons as presented in [16].The computational structures and communicational complexity of the applications written wppl** are extensively analyzed to arrive at appropriate types of functional units and its respective count, the NOC class and other related parameters as isolated entities rather than a core based form as mentioned previously. Based on statistical analysis of different types of functional units, register banks and buffers a reduced solution space is achieved.

The size of solution space is directly influenced by the system parameters(Different architectural components) and the count of individual components present in the initial architectural solution. The Architecture of ALFUs which is either serial, parallel, pipelined or parallel pipelined and the number of router levels are also be considered as system parameters which influences the size of solution space with respect to computation time and communication bandwidth. The advantage and disadvantages of different optimization algorithms like particle swarm optimization, genetic algorithm, simulated annealing and ant colony optimization are clearly explained based on their individual characteristics without any reference to many core architecture and its parameters in[19].

D. Architecturally fine grained Optimization Engine:

The optimizer engineis not a part of optimization design space which is depicted in Fig.1.The optimizer engine receives the simulated version of the initial architecture from the simulator along with information regarding resource utilization. The resource utilization refers to number of times a particular functional unit(serial or pipelined etc.) has been used over a number of clock cycles, Cache hit, the current size of cache of line, Buffer utilization at different router levels of On-Core-Level, scheduling rate,occupied pipeline stages of different functional units etc. It is stressed in the introduction section the need for architecturally fine grained optimization considering the most important aspect of high performance computing being power. Refer Fig.6.This is discussed in the next section along with the simulator results. To the best of our knowledge, no published paper exemplifies this aspect.

VIII. Simulationand Optimization Results and Analysis: many-core co-design spaces Vs traditional approaches

To establish an improved power- performance efficiency of the many-core co-design binding methodologies over the traditional, extensive optimization followed by simulations were carried out using complex applications, specifically designed for this purpose. In the companion paper II [paper 2] a customizable methodology for complex workload design and generation is presented giving number of case studies. The advantages of the proposed methodology over the existing benchmarks like SPLASH-2[],SPEC[ ],LINPACK[ ],PARSEC[ ] are also discussed. The fundamental idea behind benchmarking either an individual system , benchmarking several systems for comparative analysis, to evaluate newly

designed architecture for overall performance, evaluating the cache system ,on-core network or inter-core network, should be based on customizable workloads .In the companion paper [paper2] a systematic approach for designing such customizable workloads is presented. This innovative approach benefit the workload generation with the computation, communication, control flow complexities and the locality of reference, each under desired distribution. Another important feature in the proposed methodology is either a computational surge, a communicational surge, a control flow surge, a random surge in the locality of reference or a combination of all can be easily built into the workloads. Such wide variations in workload generations will greatly help in benchmarking either individual components of architecture or a comprehensive benchmarking for evaluation overall performance.

A. Simulation and Optimization of Target Architecture:For Multiple Workload Applications

Four complex applications were designed using the methodology in the companion paperII[paper2].The graph theoretic model of these workload applications are shown in Figs. 12a,12b,12c and 12d which are run simultaneously without space time-sharing on the xyz simulator

B. A model to showhow results are collected and analysis-carried out:

Considering the massiveness of the results presented, a self-explanatory flow diagram is shown in figure[big diagram-2] which highlights the type of results obtained in different stages the type of results obtained from simulation.

To analyze the advantages of Co-Design binding, the following combinations of ALFU based (Binding perspective)applications and ALU based(non binding perspective) applications are obtained for n varying from 4-0 while K ranges from 0-4.

No of ALFU based applications=4
No of ALU based application=0
ALFU:ALU =4:0

Similarly the results are obtained for ALFU:ALU =3:1, ALFU:ALU =2:2 ALFU:ALU =1:3 and ALFU:ALU =0:4

The ALFUvs ALU combinations are simulated using different core types. The corresponding core types under components for each of these combinations are obtained from design automation described in[3].

For the above combinations, the following results of concepts of [paper1] are obtained using simulator xyz and analyzed for both binding, binding without andnon binding perspective:

C. Co-design for co-applications Vs traditional approach:

Multi-programming Vs ALLDE

1) Co-applications aware memory bank allocation
2) Co–applications aware Cache replacement policies Vs Traditional Cache replacement policies.

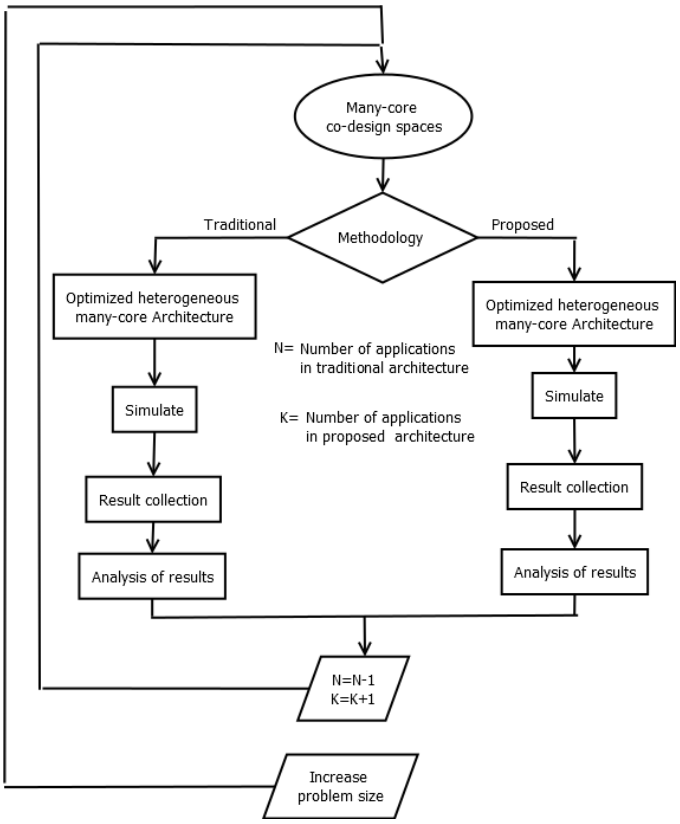

Fig. 9. *Result section flow*

3) Cache access—ALFUs/Scalars communicating directly via On Core network
4) Cache access– ALFUs/Scalars communicating via cache
5) Co-applications aware resource utilization
6) Functional unit execution tracking and Cache coherence tracking.
7) Power and performance evaluation
8) Co-applications aware scheduling performance
9) On core and inter-core router bandwidth
10) Scalability against no of cores for binding and non binding

The actual architecture of the different core types are presented in the subsequent subsections .